

Amazon Products Analysis

A comprehensive full-stack application leveraging BERT embeddings and natural language processing to provide intelligent product search, recommendations, and analysis. This system enhances the e-commerce experience by understanding semantic meaning in queries, generating concise summaries, offering multilingual support, and predicting future price trends to help users make informed purchasing decisions.

Features

- **Semantic Search**: Search products using natural language queries
- **Product Recommendations**: Get similar product recommendations based on selected items
- **Summary Generation**: Take the product description and generate concise summary
- **Multilingual Support**: Generate summaries in both English and Urdu
- **Text-to-Speech**: Play audio of product summaries in both languages
- **Product Classification**: Automatic categorization of products
- **Predict next year Price Trend**: Predict the price trend of a product for 2025

Tech Stack

Backend (Python)

- **Flask**: Web framework for building the REST API
- **Transformers**: Hugging Face's library for BERT model implementation
- **PyTorch**: Deep learning framework for model operations
- **Pandas**: Data manipulation and CSV handling
- **gTTS**: Google Text-to-Speech for audio generation
- **googletrans**: Translation service for Urdu summaries
- **Flask-CORS**: Cross-origin resource sharing support

Frontend (React)

- **Material-UI**: Component library for the user interface
- **Axios**: HTTP client for API requests
- **React Router**: Client-side routing
- **React Icons**: Icon components
- **React Hooks**: State management and side effects

Project Structure

...

```
project/
├── client/           # React frontend
│   ├── src/
│   │   ├── components/ # React components
│   │   ├── App.js      # Main application component
│   │   ├── index.js     # Application entry point
│   │   └── package.json # Frontend dependencies
├── server.py         # Flask backend server
├── train_model.py    # BERT model training script
└── product.csv       # Product database
```

```
|—— category_mappings.json # Product category mappings
|—— requirements.txt      # Python dependencies
'''
```

Key Components

Backend Components

1. **Server (server.py)**

1. Flask web server with REST API endpoints for handling HTTP requests and responses
2. BERT model integration for semantic search, text classification, and product recommendations
3. Keepa API integration for fetching Amazon product data with token management
4. CSV file storage system for saving and retrieving product information
5. Summary generation and translation to Urdu using BERT and googletrans
6. Audio file generation for text-to-speech using Google Text-to-Speech (gTTS)
7. Price trend prediction based on historical price data analysis
8. Comprehensive error handling and logging system for debugging
9. Cross-origin resource sharing (CORS) configuration for React frontend integration
10. Token pooling system to manage API rate limits and prevent token exhaustion
11. Semantic search functionality using BERT embeddings and cosine similarity
12. Product recommendation engine that finds similar products based on descriptions
13. Category prediction using the trained BERT classifier model
14. Data export functionality for downloading product information as CSV
15. Efficient data processing and performance optimization throughout the application

2. **Model Training (train_model.py)**

1. BERT model fine-tuning for product classification using transfer learning
2. Data preprocessing pipeline for cleaning and formatting product descriptions
3. Feature extraction from product titles and descriptions
4. Label encoding and category mapping generation
5. Dataset splitting into training, validation, and test sets
6. Model training with appropriate hyperparameters and optimization
7. Evaluation metrics calculation (accuracy, precision, recall, F1-score)
8. Model checkpointing to save the best performing model
9. Category mapping serialization for inference
10. Data augmentation techniques for improved model generalization
11. Cross-validation to ensure model robustness
12. Learning rate scheduling for optimal convergence
13. Early stopping to prevent overfitting
14. Model export for deployment in the server application
15. Documentation of training process and results

Frontend Components

1. **SearchComponent**

1. Advanced semantic search interface with auto-suggestions
2. Real-time search results with dynamic filtering
3. Product card grid layout with responsive design
4. Search history tracking and recent searches
5. Category-based search filtering
6. Price range and rating filters
7. Sort options by relevance, price, and rating

8. Loading states and error handling
9. Empty state handling with helpful suggestions
10. Search analytics and user behavior tracking
11. Keyboard navigation support
12. Voice search capability
13. Search result pagination
14. Search result export functionality
15. Search preferences persistence

2. ****ProductCard****

1. Clean and modern product information display
2. Expandable/collapsible product details
3. High-quality product image gallery
4. Price history mini-chart
5. Rating and review summary
6. Stock availability indicator
7. Multi-language summary support
8. Audio playback controls with progress bar
9. Share product functionality
10. Save to favorites option
11. Quick add to cart button
12. Product specifications table
13. Related products carousel
14. Product comparison checkbox
15. Mobile-optimized layout

3. ****PriceTrendComponent****

1. Interactive price history chart with zoom and pan capabilities
2. Future price trend prediction with confidence intervals
3. Side-by-side price comparison with similar products
4. Seasonal pattern highlighting with pattern explanation
5. Customizable price alert threshold configuration
6. Detailed historical price data in tabular format
7. Color-coded price change percentage indicators
8. Data-driven "Best Time to Buy" recommendations
9. Advanced price trend filtering by time period and category
10. One-click export of price data to CSV format
11. Intuitive price history timeline with key events
12. Visual confidence indicators for prediction reliability
13. Market condition impact analysis with explanatory notes
14. Social sharing of price trend insights via email or link
15. Fully responsive design for desktop and mobile viewing
16. Price trend summary with key insights and recommendations
17. Integration with product recommendation system
18. Price history download for offline analysis
19. Custom date range selection for focused analysis
20. Price trend notifications and alerts system

4. ****NavigationComponent****

1. Responsive navigation bar with mobile menu
2. Breadcrumb navigation for deep linking
3. Quick access to recent searches
4. Category navigation dropdown
5. User preferences menu

6. Language selector
 7. Theme switcher (light/dark mode)
 8. Notification center
 9. User account menu
 10. Search bar integration
 11. Cart preview
 12. Favorites quick access
 13. Help and support links
 14. Accessibility features
 15. Keyboard shortcuts menu
5. ****RecommendationComponent****
1. Personalized product recommendations
 2. Similar products carousel
 3. "Frequently bought together" suggestions
 4. Category-based recommendations
 5. Trending products section
 6. Recently viewed items
 7. Recommendation explanation tooltips
 8. Filter recommendations by relevance
 9. Save recommendation preferences
 10. Share recommendations feature
 11. Recommendation feedback system
 12. A/B testing support
 13. Recommendation analytics
 14. Cross-sell opportunities
 15. Up-sell suggestions

Data Flow

1. ****Search Process****
 - User enters search query in the frontend
 - Query is processed and sent to backend API
 - Backend converts query to BERT embeddings
 - Similarity search performed against product database
 - Results are ranked by relevance
 - Frontend displays results in product cards
 - User can filter and sort results
2. ****Recommendation Process****
 - User selects a product of interest
 - Product details are sent to recommendation API
 - Backend generates product embeddings
 - Similarity comparison with all products
 - Top matching products are selected
 - Results are ranked by similarity score
 - Frontend displays recommendations
 - User can view detailed comparisons
3. ****Summary Generation****
 - User requests summary for a product
 - Backend retrieves product description
 - BERT model generates concise summary
 - Summary is translated to Urdu

- Audio files are generated for both languages
- Frontend displays summaries with audio controls
- User can play/pause audio in both languages

4. ****Price Trend Analysis****

- User selects product for price analysis
- Backend fetches historical price data
- Price patterns are analyzed
- Future trends are predicted
- Data is formatted for visualization
- Frontend displays interactive charts
- User can view predictions and insights

5. ****Data Storage****

- Product data stored in CSV format
- BERT embeddings cached for performance
- Audio files stored temporarily
- User preferences saved in browser
- Search history maintained locally
- Price history updated regularly
- Category mappings stored in JSON

Dependencies

Backend Dependencies

1. ****Web Framework****

- Flask (3.0.2): Lightweight web framework
- Flask-CORS (4.0.0): Cross-origin resource sharing

2. ****Machine Learning****

- PyTorch ($\geq 2.0.0$): Deep learning framework
- Transformers ($\geq 4.30.0$): BERT model implementation
- scikit-learn ($\geq 1.0.0$): Machine learning utilities
- numpy ($\geq 1.21.0$): Numerical computations

3. ****Data Processing****

- pandas ($\geq 1.5.0$): Data manipulation
- tqdm ($\geq 4.65.0$): Progress bars

4. ****Audio & Translation****

- gTTS (2.5.1): Text-to-speech conversion
- googletrans (3.1.0a0): Language translation
- sentencepiece ($\geq 0.1.99$): Text tokenization
- protobuf ($\geq 4.25.1$): Data serialization

5. ****API & Networking****

- requests (2.31.0): HTTP requests

Frontend Dependencies

1. ****Core****

- react: UI library
- react-dom: React rendering

2. ****UI Components****

- @mui/material: Material Design components
- @mui/icons-material: Material icons

3. **Data & Routing**

- axios: HTTP client
- react-router-dom: Client-side routing

4. **Machine Learning**

- @tensorflow/tfjs: Deep learning and neural networks
- chart.js: Data visualization
- react-chartjs-2: React wrapper for Chart.js

Setup and Installation

1. **Prerequisites**

- Python 3.8 or higher
- Node.js 14 or higher
- npm 6 or higher
- Git

2. **Backend Setup**

```

``bash
# Clone the repository
git clone <repository-url>
cd project2

# Create and activate virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install Python dependencies
pip install -r requirements.txt

# Train the BERT model
python train_model.py

# Start the Flask server
python server.py
```

```

### 3. **Frontend Setup**

```

``bash
Navigate to client directory
cd client

Install Node.js dependencies
npm install

Start the development server
npm start
```

```

4. **Configuration**

- Set up environment variables in `.env` file

- Configure API keys for external services
- Adjust server settings in `config.py`
- Set up database connections

5. ****Verification****

- Backend server running on <http://localhost:5004>
- Frontend development server on <http://localhost:3000>
- Test API endpoints
- Verify model training
- Check database connections